

Lab 5: MQTT in IOT Applications

Author: Andrew J Napieraj

Class: ESCE 6660 Internetworking of Things (IoT)

Instructor: Prof. Ish Jain

TA: Chibuikem Ezemaduka

Due Date: February 2, 2026

Part 1: Initial MQTT Questions

1. Why is MQTT built upon TCP?

- a. Reliability: TCP retransmits lost packets. MQTT is often used for telemetry, commands, and device control, where losing a message can matter.
- b. Ordering: TCP preserves byte order. That helps MQTT maintain the correct sequence of control packets and message flow.
- c. Connection-oriented: MQTT uses a long-lived client–broker session. TCP naturally supports that model.
- d. Built-in flow and congestion control: TCP already handles network pacing, so MQTT can stay lightweight instead of reinventing that logic.
- e. QoS support is easier: MQTT has QoS 0, 1, and 2. Those reliability levels are much easier to implement on top of a transport that is already reliable and stateful.

Why not UDP? MQTT would become more complex and lose part of its main advantage: simple lightweight messaging. UDP is:

- a. Connectionless
- b. Unordered
- c. unreliable by default
- d. no retransmission, no acknowledgment, no congestion control

If MQTT were built on UDP, MQTT itself would have to add much more machinery to handle:

- a. packet loss
- b. reordering
- c. duplicate suppression
- d. retransmission timing
- e. session tracking

2. What are some advantages and drawbacks of using MQTT in IoT? List at least 2 for each.

Advantages of MQTT in IoT:

Low overhead / lightweight protocol by using a small header and minimal control traffic, it works well on low-bandwidth networks and resource-constrained devices.

Efficient for many devices with its publish/subscribe model, it lets many sensors and clients communicate through a broker without needing direct device-to-device connections.

Reliable delivery options through supports QoS levels (0, 1, 2), so the system can choose between fastest delivery and more reliable delivery depending on the application.

Good for unreliable networks where connections may be intermittent, since clients can reconnect and continue communicating through the broker.

Drawbacks of MQTT in IoT:

Requires a broker which adds infrastructure and can become a single point of failure if not designed redundantly.

TCP overhead and therefore has more transport overhead than UDP-based approaches, which may matter for very constrained or very low-power systems.

Not ideal for large payloads like transferring large files or heavy data streams.

Security must be added carefully because MQTT itself is lightweight, strong security usually requires extra mechanisms like TLS, authentication, and access control, which add complexity to the overall system.

3. What are some suitable IoT scenarios for applying MQTT? List at least 2.

Smart home systems such as sensors, lights, thermostats, and door locks that can send small status updates and receive commands through a broker.

Industrial monitoring and control for factory machines and sensors that can publish temperature, pressure, vibration, or fault data to a central monitoring system.

Remote environmental sensing for weather stations, soil sensors, or water-level monitors that can send periodic telemetry over limited-bandwidth links.

Fleet or vehicle telemetry vehicles can publish location, health, and diagnostic data back to a server in near real time.

Part 2: IOT Application

Overall, setup for this project was easy to follow, with some initial errors as identified by the Lab notes. The figures below show all the deliverables from the lab.

1. Provide screenshots of the code portion you modified. One for Lab5_PI.py and another for Lab5_PC.py. Code at end (snippets edited only).
2. Provide screenshots of the RPi terminal and the PC terminal of successfully checking multiple people.

```
(env) andre@DrewShoo:~/lab5$ python Lab5_PC.py
/home/andre/lab5/Lab5_PC.py:8: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  self.client = mqtt.Client(client_id=CLIENT_ID, clean_session=True, userdata=None, protocol=mqtt.MQTTv311, transport="tcp", callback_api_version=mqtt.CallbackAPIVersion.VERSION1)
Connected to test.mosquitto.org port: 1883
Flags: {'session present': 0} returned code: 0
todd checked in on scanner 1 at 2026-04-05 12:22:48
todd checked in on scanner 1 at 2026-04-05 12:27:53
todd checked in on scanner 1 at 2026-04-05 12:28:18
todd checked in on scanner 1 at 2026-04-05 12:29:40
andre checked in on scanner 1 at 2026-04-05 12:30:46
```

Figure 1: From PC

```
File Edit Tabs Help
File "/home/napiea/Documents/env/lib/python3.13/site-packages/mfr522/MFR522.py", line 217, in MFR522_ToCard
  n = self.Read_MFR522(self.ComIrqReg)
File "/home/napiea/Documents/env/lib/python3.13/site-packages/mfr522/MFR522.py", line 162, in Read_MFR522
  val = self.spi.xfer2([((addr << 1) & 0x7E) | 0x80, 0])
KeyboardInterrupt

(env) napiea@napiea:~/Documents $ python write.py
New data:todd
New place your tag to write
Written
(env) napiea@napiea:~/Documents $ python Lab5_PI.py
/home/napiea/Documents/Lab5_PI.py:13: DeprecationWarning: Callback API version 1 is deprecated, update to latest version
  self.client = mqtt.Client(client_id=CLIENT_ID, clean_session=True, userdata=None, protocol=mqtt.MQTTv311, transport="tcp")
Connected to test.mosquitto.org port: 1883
Flags: {'session present': 0} returned code: 0
todd's check-in at scanner 1 authenticated with uid: 0a9bca4ee4644531b49e933b29c0a04e
todd's check-in at scanner 1 authenticated with uid: 92091137047742edad8d99c902aba063
todd's check-in at scanner 1 authenticated with uid: 02b0c5f997874db6aad546d226f0da28
todd's check-in at scanner 1 authenticated with uid: 078c14148664a119a20e1945d0a1149
andre's check-in at scanner 1 authenticated with uid: adf2e9670e74de7b50ed2148fc9b70r
```

Figure 2: From RPi

3. Provide screenshot of the Web UI

Smart Attendance

Check-In

Scanner	Name	Timestamp	Assigned UID	Total: 2
1	todd	2026-04-05 12:29:40	9650e314b34440b08f705334a1b740e	
1	andre	2026-04-05 12:30:46	adf2e9670e74de7b50ed2148fc9b70f	

ANDRE/MFRC

connected

[ws] /ws/timestamp

connected 1

ANDRE/UID

connected

[ws] /ws/uid

connected 1

Figure 3: From User Interface

- Use Wireshark on the PC to capture an MQTT PUBLISH packet during the communication. Take screenshots of the packet with MQTT details expanded and fill in the following table:

Table 1 Packet analysis of a MQTT PUBLISH packet using Wireshark

MQTT Section	Bit Range (start from 0)	Which fields are in the section and what are their values? (provide screenshots as evidence)
Fixed Header	0-15	Byte 1 (bits 0–7): Control Packet Type = PUBLISH (3), DUP = 0 (not set), QoS = 0 (“at most once delivery” / fire-and-forget), Retain = 0 (not set). Byte 2 (bits 8–15): Remaining Length = 38 bytes.
Variable Header	16-111	Topic Length (occupies 2 bytes) = says topic is length of 10 bytes, Topic Name = ANDRE/MFRC (10 bytes). Since QoS = 0, there is no Packet Identifier field in this packet.
Payload	112-319	Message field = 1#todd#2026-04-05 12:29:40. Payload length = 26 bytes.

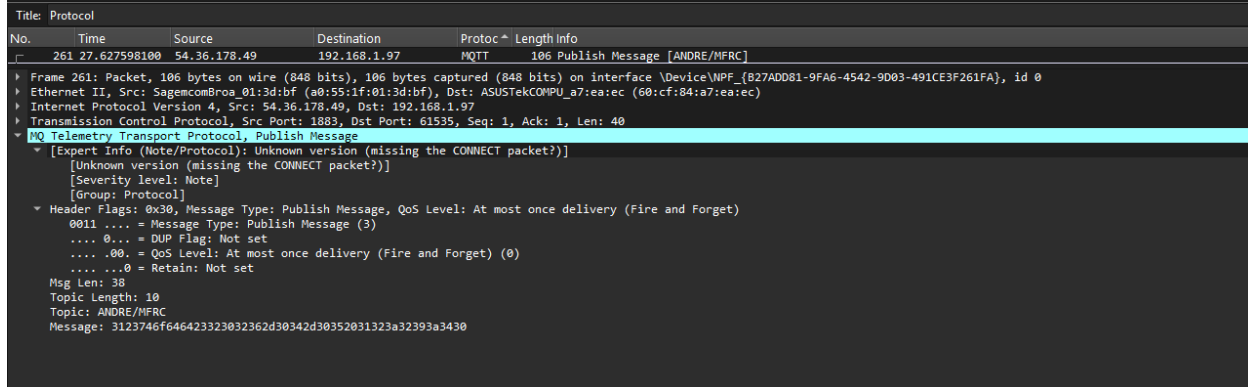


Figure 4: MQTT Packet Info

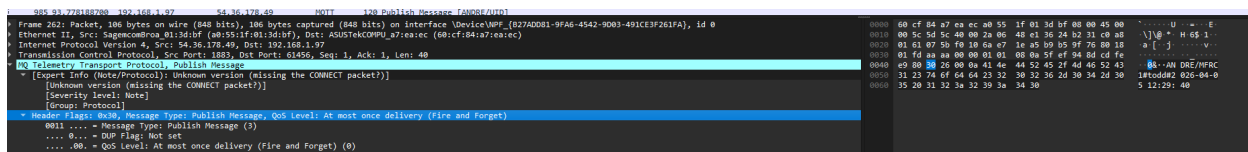


Figure 5 Fixed Header 1st Byte

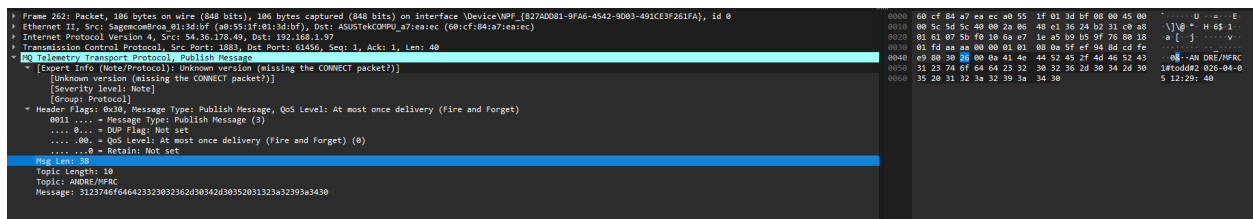


Figure 6: Fixed Header Second Byte

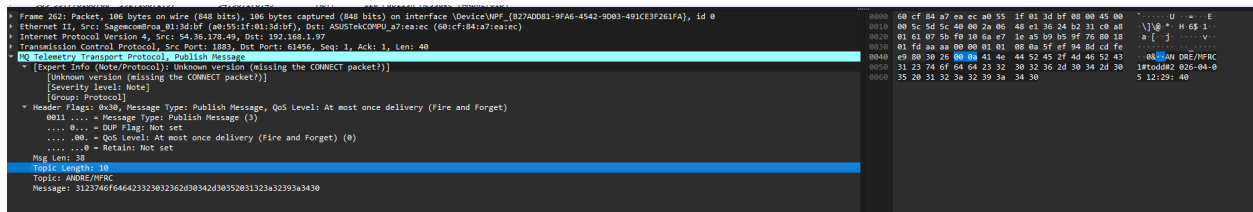


Figure 7: Topic Length (Two Bytes)

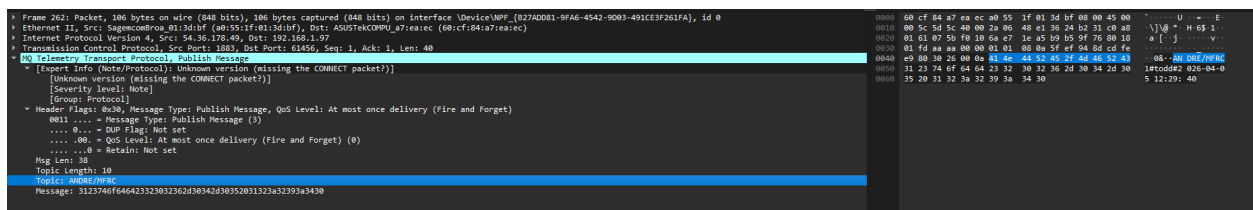


Figure 8: Topic (10 Bytes)

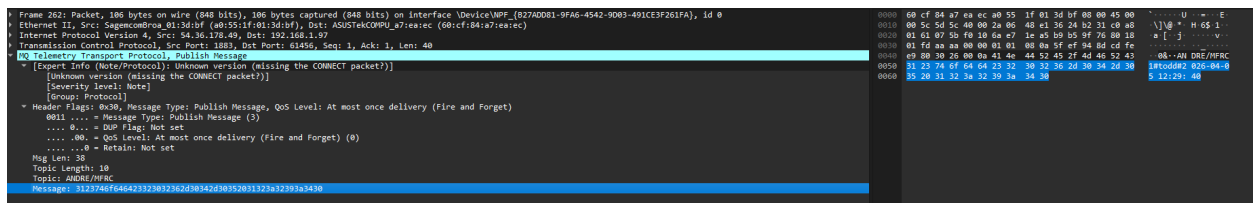


Figure 9: Payload (26 Bytes)

From the captured MQTT PUBLISH packet, the fixed header occupies bits 0–15 and contains packet type PUBLISH, DUP=0, QoS=0, Retain=0, and Remaining Length=38 bytes. The variable header occupies bits 16–111 and contains Topic Length=10 and Topic Name=ANDRE/MFRC. Because the QoS level is 0, no Packet Identifier is present. The payload occupies bits 112–319 and contains the message 1#todd#2026-04-05 12:29:40. The decoded message is seen in *Table 1*.

5. What is the function of each field in each section?

Fixed Header

This section tells the receiver what kind of MQTT control packet it is and how to interpret the rest of the packet.

1. Control Packet Type = PUBLISH

- Identifies this packet as an MQTT PUBLISH message.
- Its purpose is to carry application data from a publisher to a broker or subscriber.

2. DUP flag = 0

- Indicates whether this is a retransmission of an earlier PUBLISH packet.
- A value of 0 means this is not marked as a duplicate transmission.

3. QoS level = 0

- Defines the delivery guarantee.
- QoS 0 means at most once delivery, so the message is sent without acknowledgment and may be lost if an error occurs.

4. Retain flag = 0

- Tells the broker whether it should store this message as the last known value for the topic.
- A value of 0 means the broker should not retain it.

5. Remaining Length = 38

- Gives the number of bytes that follow the fixed header.
- It lets the receiver know how much data remains in the packet.

Variable Header

This section contains metadata needed to process the PUBLISH message.

1. Topic Length = 10

- Indicates the number of bytes in the topic name.
- This helps the receiver know where the topic field ends.

2. Topic Name = ANDRE/MFRC

- Specifies the topic to which the payload belongs.
- The broker uses this to route the message to any clients subscribed to that topic.

3. Packet Identifier

- Not present here because the message uses QoS 0.
- This field would only be needed for QoS 1 or QoS 2 to track acknowledgments and reliable delivery.

Payload

This section contains the actual application data being sent.

1. Message = 1#todd#2026-04-05 12:29:40

- This is the user/application content of the MQTT message.
- It is the data that the publisher wants the receiver or broker to process.

6. From your capture, what is the IP address and port number of your broker?

From the selected packet:

- Source IP: 54.36.178.49
- Destination IP: 192.168.1.97
- TCP Src Port: 1883
- TCP Dst Port: 61456

Since port 1883 is the standard MQTT broker port, the broker is:

- Broker IP address: 54.36.178.49
- Broker port: 1883

PC is the client at:

- Client IP address: 192.168.1.97
- Client ephemeral port: 61456

7. Suppose your Raspberry Pi or PC is unable to connect to the broker. Across the network, transport, and application layers, what key diagnostic steps would you take to identify the issue? Be as specific as possible, including the exact commands you would use.

Troubleshoot this layer by layer.

1. Network layer checks

These verify that the client has an IP address, can reach the local network, and can route traffic to the broker.

a. On Windows PC (command prompt)

- i. Check IP configuration:

```
ipconfig /all
```

Things to verify:

- The PC has a valid IPv4 address
- Subnet mask is correct
- Default gateway is present
- DNS server is present if using a hostname

- ii. Test reachability to the local gateway:

```
ping 192.168.1.1
```

- iii. Test reachability to the broker:

```
ping 54.36.178.49
```

- iv. Trace the route to the broker:

```
tracert 54.36.178.49
```

- v. Check ARP table for local resolution:

```
arp -a
```

b. On Raspberry Pi / Linux (cmd)

- i. Check interface and address:

```
ip addr  
ip route
```

- ii. Test gateway:

```
ping -c 4 192.168.1.1
```

- iii. Test broker:

```
ping -c 4 54.36.178.49
```

- iv. Trace route:

```
traceroute 54.36.178.49
```

If traceroute is not installed:


```
sudo apt install traceroute
```

If ping fails to the gateway, the issue is likely local network configuration. If ping works locally but not to the broker, the problem is probably routing, upstream internet access, or broker reachability.

2. Transport layer checks

These verify whether TCP connectivity to port 1883 is working.

a. On Windows

- i. Test whether TCP port 1883 is reachable (powershell):

```
Test-NetConnection 54.36.178.49 -Port 1883
```

- ii. Check for existing TCP sessions:

```
netstat -ano | findstr 1883
```

b. On Raspberry Pi / Linux

- i. Test TCP port with netcat:

```
nc -vz 54.36.178.49 1883
```

- ii. Or with telnet:

```
telnet 54.36.178.49 1883
```

If telnet is missing:

```
sudo apt install telnet
```

- iii. Check socket states:

```
ss -tan | grep 1883
```

If TCP port 1883 is closed or filtered, likely causes are:

- Broker not running
- Firewall blocking inbound traffic
- ISP or network policy blocking the port
- Wrong IP address or port

3. Application layer checks

These verify that MQTT itself is working, not just TCP.

a. On Linux / Raspberry Pi/ PC:

i. Publish test with mosquitto client

```
mosquitto_pub -h 54.36.178.49 -p 1883 -t test/topic  
-m "hello"
```

ii. Subscribe test:

```
mosquitto_sub -h 54.36.178.49 -p 1883 -t test/topic
```

iii. Verbose connection output:

```
mosquitto_sub -h 54.36.178.49 -p 1883 -t test/topic  
-d
```

Or:

```
mosquitto_pub -h 54.36.178.49 -p 1883 -t test/topic  
-m "hello" -d
```

iv. If authentication is required:

```
mosquitto_sub -h 54.36.178.49 -p 1883 -t test/topic  
-u username -P password -d
```

v. If TLS is required instead of plain MQTT:

```
mosquitto_sub -h 54.36.178.49 -p 8883 -t test/topic  
--cafile ca.crt -u username -P password -d
```

b. Broker-side checks on the host running Mosquitto

i. With access to the broker machine, verify the service is running:

```
sudo systemctl status mosquitto  
sudo systemctl restart mosquitto
```

ii. Check that it is listening on port 1883:

```
sudo ss -tlnp | grep 1883
```

iii. Read broker logs:

```
sudo journalctl -u mosquitto -n 50
```

Or:

```
sudo tail -f /var/log/mosquitto/mosquitto.log
```

iv. Check firewall rules:

```
sudo ufw status
sudo iptables -L -n
```

c. Packet capture checks

- i. Capture connection attempts to confirm what is happening:

```
sudo tcpdump -i any port 1883
```

Or in Wireshark use:

```
tcp.port == 1883
```

What to look for:

- TCP 3-way handshake: SYN, SYN-ACK, ACK
- MQTT CONNECT from client
- MQTT CONNACK from broker

If you see SYN packets but no SYN-ACK, transport connectivity is failing. If TCP connects but no CONNACK returns, the application layer is failing.

8. If you do not want your broker to be hosted on the internet, what changes would you make to its IP address to ensure the MQTT service continues to function correctly? Provide an example of a suitable IP address based on your current network configuration.

The public IP is 54.36.178.49. That means it is internet-reachable. To place the broker on a private LAN address instead and have local clients connect to that private IP select a private IPv4 range in the subnet 192.168.1.x, for example:

- **Example broker IP:** 192.168.1.50
- **Broker port:** 1883

That would let the MQTT service continue working for devices on the local network while no longer requiring the broker to be exposed on the public internet.

Typically for private LAN designs, they should bind Mosquitto to the LAN interface only, avoid port forwarding on the router, and update clients to connect to 192.168.1.50 instead of 54.36.178.49.

PC

```
# The callback for when the client receives a CONNACK response from the
server.
def on_connect(self, client, userdata, flags, rc):
    client.subscribe("ANDRE/MFRC", qos=0)
# The callback for when a message is received.
def on_message(self, client, userdata, msg):
    if msg.topic == "ANDRE/MFRC":
        client.publish("ANDRE/UID", payload="#".join([pi_number, name,
uuid.uuid4().hex]), qos=0, retain=False)
```

RPi

```
# The callback for when the client receives a CONNACK response from the
server.
def on_connect(self, client, userdata, flags, rc):
    print("Connected to ", client._host, "port: ", client._port)
    print("Flags: ", flags, "returned code: ", rc)
    client.subscribe("ANDRE/UID", qos=0)

# The callback for when a message is received.
def on_message(self, client, userdata, msg):
    message = str(msg.payload.decode("utf-8"))
    if msg.topic == "ANDRE/UID":

    def start_tracking(self):
        self.client.publish("ANDRE/MFRC",
payload="#".join([self.pi_number, name, timestamp]), qos=0, retain=False)
```